

BEJSON CMS: TECHNICAL TAXONOMY & NAMING STANDARD REPORT

ARCHITECTURAL BLUEPRINT FOR SYSTEMIC FIELD NAMING, UNIFIED UUID IDENTIFIERS, AND PAGE TYPE EXTENSIBILITY

Author: Elton Boehnen (boehnenelton2024@gmail.com)
Project Target: BEJSON_CMS v18.28 (Package 81)
Document Version: 1.0.0 (Package Version: 102 | RELATIONAL_ID: 9b0a3d8f-5c2e-4e9b-af0d-8e6b2c0a4f3e)
Status: Architectural Proposal & Migration Plan

1. EXECUTIVE SUMMARY

The BEJSON CMS ecosystem is a high-performance static site generator and web publishing platform built on Flask, top-level BEJSON schemas, and MFDB binary containers across 84 core Python modules. Over extensive feature iterations, legacy shortcuts have introduced field key fragmentation across database files, Flask blueprints, and rendering templates. For instance, entity references present ad-hoc names such as `ad_link` vs `external_url`, `auth_name` vs `author_ref`, and `featured_img` vs `app_image` vs `ad_image`.

As BEJSON CMS expands to support rich, specialized Page Types—including standard HTML pages, dedicated video hubs, and document download centers with PDF media attachments—integrating new features against messy variable names introduces severe maintenance overhead and risk of schema corruption.

This report delivers a rigorous technical architecture to unify field naming across the codebase. It establishes:

1. An **Empirical Audit** of all 14 major divergent field names across 84 source files.
2. A **Canonical Prefix Registry** enforcing explicit entity tags (e.g., `page_`, `author_`, `asset_`, `extmedia_`).
3. A **Unified Synthetic UUID Standard** using `<prefix>_uuid` primary keys to eliminate fragile name-keying.
4. A **Polymorphic Page Type Extensibility Pattern** supporting Standard, Video, and Document variants via strategy renderers.
5. A **Phased Migration Roadmap** with effort estimates for zero-downtime execution.

2. EMPIRICAL AUDIT OF CURRENT FIELD USAGE & FRAGMENTATION

An automated static analysis of the `src/` directory across all web blueprints (System, Content, Media, Interface), shared utility libraries, and CLI tools identified 14 key fields suffering from semantic overlap, prefix ambiguity, or missing entity context:

Field Name	File Count	Occurrences	Current Domain / Architectural Issues
<code>created_at</code>	12 files	32	Generic timestamp key; collides with <code>uploaded_at</code> and lacks entity prefix
<code>featured_img</code>	8 files	55	Abbreviated image reference; overlaps with <code>app_image</code> and <code>ad_image</code>
<code>category_ref</code>	8 files	36	Foreign key using <code>_ref</code> suffix string instead of standardized Category UUID
<code>author_ref</code>	8 files	28	Foreign key storing author display name string rather than persistent Author UUID
<code>item_type</code>	8 files	24	Un-prefixed type discriminator string; lacks namespace control
<code>external_url</code>	7 files	20	Inconsistent with <code>ad_link</code> and embedded media link keys
<code>uploaded_at</code>	6 files	13	Entity-specific upload timestamp used only in media assets
<code>auth_name</code>	6 files	40	Truncated author name key in persona/profile schema
<code>entry_file</code>	5 files	31	Storage file path pointer lacking entity prefix context
<code>ad_link</code>	4 files	7	Domain-specific advertisement target URL; duplicate concept of <code>external_url</code>
<code>app_image</code>	4 files	17	System image asset key; conflicts with <code>featured_img</code>
<code>ad_image</code>	4 files	9	Ad banner asset reference; un-prefixed image key

auth_img	3 files	14	Truncated author avatar image reference key
media_uuid	3 files	16	Prefix collision point between local MediaAsset and ExternalMedia

Core Technical Debt Patterns Identified:

- **Domain Abbreviation Fragmentation:** Field names routinely drop entity prefixes or use truncated shorthand (e.g., auth_name, auth_img) leading to ambiguous variable scope in Python functions.
- **Fragile String-Based Foreign Keys:** Relationships between pages, categories, and authors rely on mutable strings (e.g., author_ref = 'Elton Boehnen') which break upon display name updates or slug changes.
- **Prefix Collision Risks:** Both local media uploads and third-party video embeds share the media_ prefix (e.g. media_uuid), making search indexing and renderer lookup logic collision-prone.

3. CANONICAL PREFIX REGISTRY & NAMING MANDATES

Under the BEJSON 104a Core Mandate, positional indexing is strictly prohibited—all record access must proceed via the Field Map Cache. To enforce global clarity across all schemas, every field name must strictly follow full snake_case prefixed by its canonical entity tag.

Canonical Prefix Lookup Table:

Entity Name	Canonical Tag	Primary Key Standard	Standardized Field Mapping Examples
Page Content	page_	page_uuid	page_title, page_slug, page_type, page_created_at
Author Profile	author_	author_uuid	author_display_name, author_avatar_asset_uuid, author_bio
Media Asset	asset_	asset_uuid	asset_file_name, asset_mime_type, asset_file_size_bytes
External Media	extmedia_	extmedia_uuid	extmedia_platform, extmedia_embed_url, extmedia_aspect_ratio
Ad Placement	ad_	ad_uuid	ad_title, ad_target_url, ad_banner_asset_uuid, ad_slot_id
Taxonomy Category	cat_	cat_uuid	cat_name, cat_slug, cat_description, cat_parent_uuid
Taxonomy Tag	tag_	tag_uuid	tag_name, tag_slug
System Setting	sys_	sys_uuid	sys_key, sys_value, sys_updated_at

Collision Resolution Rules:

1. **Local MediaAsset vs ExternalMedia Collision:** Local binary uploads retain asset_ (e.g. asset_uuid, asset_file_name), while remote embeds utilize extmedia_ (e.g. extmedia_uuid, extmedia_embed_url). This completely eliminates key overlap.
2. **Author Profile Synthetic UUID:** AuthorProfile introduces author_uuid as primary key. The legacy auth_name string key is replaced by foreign key page_author_uuid.

4. UNIFIED SYNTHETIC UUID & BEJSON 104a SCHEMA SPECIFICATION

All entities must include a synthetic RFC 4122 Version 4 UUID formatted as <prefix>_uuid. Relying on natural keys (such as page slugs or author names) is strictly prohibited as primary identifiers because natural keys change over time and fail under soft-delete or localization constraints.

BEJSON 104a Standardized Page Content Schema:

```
{
  "Format": "BEJSON",
  "Format_Version": "104a",
  "Format_Creator": "Elton Boehnen",
  "Session_Id": "f6141ec9-6a9e-4515-ba8a-987b43bfb052",
  "Project_Name": "BEJSON_CMS",
  "Schema_Version": "1.6.0",
  "Records_Type": ["PageContent"],
  "Fields": [
    {"name": "page_uuid", "type": "string"},
    {"name": "page_title", "type": "string"},
    {"name": "page_slug", "type": "string"},
    {"name": "page_type", "type": "string"},
    {"name": "page_category_uuid", "type": "string"},
    {"name": "page_author_uuid", "type": "string"},
    {"name": "page_featured_asset_uuid", "type": "string"},
    {"name": "page_created_at", "type": "string"},
    {"name": "page_updated_at", "type": "string"}
  ],
  "Values": [
    [
      "a3b8c9d1-4e5f-6a7b-8c9d-0e1f2a3b4c5d",
      "Building Scalable BEJSON Taxonomies",
      "building-scalable-bejson-taxonomies",
      "standard",
      "c1a2b3c4-d5e6-7f8a-9b0c-1d2e3f4a5b6c",
      "e5f6a7b8-c9d0-1e2f-3a4b-5c6d7e8f9a0b",
      "f9e8d7c6-b5a4-3f2e-1d0c-9b8a7f6e5d4c",
      "2026-08-10T04:27:00Z",
      "2026-08-10T04:27:00Z"
    ]
  ]
}
```

5. PAGE TYPE EXTENSIBILITY ARCHITECTURE

To prevent schema pollution with sparse, type-specific columns, BEJSON CMS employs a **Polymorphic Variant Metadata Pattern**. The core PageContent schema handles common attributes, while extended attributes live in specialized sub-record schemas linked by page_uuid.

Page Type Specifications:

- 1. **Standard HTML Page (page_type = 'standard')**: Standard articles with HTML content stored in page_body_html.
- 2. **Video Hub Article (page_type = 'video')**: Attaches a PageVideoMetadata record containing video embed URLs, duration, provider tags, and transcript asset UUIDs.
- 3. **Document Hub Page (page_type = 'document')**: Attaches a PageDocumentMetadata record linking primary PDF/EPUB asset UUIDs, version strings, byte sizes, and download rules.

Python Strategy Renderer Pattern:

```
# Python Renderer Strategy Pattern for Polymorphic Page Types
from typing import Dict, Any

class PageTypeRenderer:
    def render(self, page_data: Dict[str, Any], meta_data: Dict[str, Any]) -> str:
        raise NotImplementedError

class StandardPageRenderer(PageTypeRenderer):
    def render(self, page_data: Dict[str, Any], meta_data: Dict[str, Any]) -> str:
        return f"<article class='page-standard'><h1>{page_data['page_title']}</h1>{page_data['page_body_html']}</article>"

class VideoPageRenderer(PageTypeRenderer):
    def render(self, page_data: Dict[str, Any], meta_data: Dict[str, Any]) -> str:
        embed_url = meta_data.get('video_embed_url', '')
        return f"""<article class='page-video'>
            <h1>{page_data['page_title']}</h1>
            <div class='video-player'><iframe src='{embed_url}'></iframe></div>
            <div>{page_data['page_body_html']}</div>
        </article>"""

class DocumentPageRenderer(PageTypeRenderer):
    def render(self, page_data: Dict[str, Any], meta_data: Dict[str, Any]) -> str:
        doc_uuid = meta_data.get('doc_asset_uuid', '')
        return f"""<article class='page-document'>
            <h1>{page_data['page_title']}</h1>
            <div class='doc-download-card'>
                <p>PDF Document ({meta_data.get('doc_file_size_formatted', '')})</p>
                <a href='/assets/download/{doc_uuid}' class='btn btn-primary'>Download PDF</a>
            </div>
        </article>"""

RENDERER_REGISTRY = {
    'standard': StandardPageRenderer(),
    'video': VideoPageRenderer(),
    'document': DocumentPageRenderer()
}
```

6. IMPLEMENTATION ROADMAP & MIGRATION PHASES

Migrating existing storage containers and 84 Python source files requires a structured 4-phase rollout to guarantee data safety and zero publisher downtime:

Phase	Focus & Deliverables	Target Modules	Estimated Effort
Phase 1	Prefix Registry & Alias Adapter Layer: Build BEJSON_Taxonomy.py lookup maps. Support dual reads for legacy & new keys.	src/lib/lib_bejson_core.py src/lib/BEJSON_Taxonomy.py	12 Hours (1-2 Days)
Phase 2	Data Migration Scripting: Execute batch conversion on all storage/mfdb/ containers. Inject synthetic UUIDs across all records.	storage/mfdb/*.json tools/migrate_taxonomy.py	16 Hours (2 Days)
Phase 3	Publisher & Blueprint Refactoring: Update BEJSON_CMS_Publisher.py, Content, System, Media, Interface blueprints to use new field map keys.	src/web/BEJSON_CMS_*.py resources/templates/*.html	24 Hours (3 Days)
Phase 4	Page Type Engine & Documentation: Deploy Video/Document rendering strategies and produce updated API spec docs.	src/web/BEJSON_CMS_Renderers.py docs/taxonomy_guide.md	16 Hours (2 Days)

7. CONCLUSION & SIGN-OFF

By implementing the **Canonical Prefix Registry**, instituting standard <prefix>_uuid identifiers, and deploying polymorphic strategy renderers, BEJSON CMS eliminates legacy naming debt and establishes a resilient foundation for advanced media and document publishing capabilities.

Report Prepared By: Elton Boehnen	Contact: boehnenelton2024@gmail.com
Portfolio: boehnenelton2024.pages.dev	GitHub: github.com/boehnenelton